GraphQL & Next.js

Assessment

Theme: Creator Portfolio Builder

Section A: Conceptual Understanding

Q.1 You are building a creator portfolio where profile data, projects, and social links render dynamically. Explain how JSX, component composition, and props work together to keep the UI reusable and consistent.

Ans.

- JSX (JavaScript XML) is a syntax extension used in React and Next.js that allows developers to write HTML-like code inside JavaScript. It makes UI development simple and readable.
- Component composition means dividing the application into small reusable components such as Profile, ProjectCard, and SocialLinks. These components can be combined to build a complete user interface.
- Props (properties) are used to pass data from a parent component to a child component. For example, a ProjectCard component can receive the project title, description, and year through props and display them dynamically.
- By using JSX, component composition, and props together, developers can create reusable and consistent UI components. The same component can display different data without rewriting code, making the application easier to maintain and scale.

Q.2 The portfolio supports live updates when a creator edits details. Explain how state and event handling enable controlled updates without reloading the page.

Ans.

- State is used to store dynamic data such as the creator's name, bio, and project details. When a user edits information, the state is updated instead of reloading the entire page.
- Event handling captures user actions such as clicking a button or typing in an input field. Functions like onChange and onClick trigger state updates.
- When the state changes, React automatically re-renders only the affected components, providing instant updates on the screen. This creates a smooth user experience without requiring a page refresh.

Q.3 Some sections should appear only if data exists (e.g., Projects). Explain how conditional rendering and list rendering manage this behavior efficiently.

Ans.

- Conditional rendering allows components to be displayed only when certain conditions are true. For example, if the projects array contains data, the project section is displayed; otherwise, a message like "You haven't added any projects yet" is shown.
- List rendering uses the map() function to display multiple items from an array. Each item is rendered as a separate component and should have a unique key property for efficient updates.
- Using conditional rendering and list rendering together improves performance and ensures that only relevant content is displayed to the user.

Q.4 Profile data should persist across reloads. Explain how LocalStorage or API-based persistence integrates with component state.

Ans.

- LocalStorage is a browser feature used to store data permanently on the user's device. When profile information changes, it is saved to LocalStorage.
- When the application starts, the stored data is retrieved and loaded into the component state. This ensures that the profile information remains available even after refreshing the page.
- In API-based persistence, the data is stored on a server instead of the browser. The application sends updates through API or GraphQL mutations and retrieves the latest data using API calls or GraphQL queries, allowing access from multiple devices.

Q.5 As the app grows, explain how breaking the UI into smaller components improves maintainability and performance.

Ans.

- Breaking the UI into smaller components makes the code more organized and easier to understand. Each component has a single responsibility and can be developed, tested, and maintained independently.
- Reusable components reduce code duplication and improve consistency across the application. React also updates only the components whose data has changed, improving rendering performance.
- This modular approach simplifies debugging and allows multiple developers to work on different parts of the project simultaneously.

Q.6 Explain how this component-based architecture prepares the app for future scaling or migration to Next.js.

Ans.

- A component-based architecture separates the application into reusable and independent modules. This makes it easy to add new features or modify existing functionality without affecting the entire project.
- Since Next.js is built on React, existing components can be reused with minimal changes. The application can also take advantage of Next.js features such as server-side rendering (SSR), static site generation (SSG), routing, and performance optimization.
- As the application grows, this architecture supports better scalability, maintainability, and faster development while providing an improved user experience.

Section B: Mini Tasks - Creator Portfolio Builder

Task: Portfolio Item Manager

Q.1 Reusable ProjectCard Component: Build a component that displays a Project's Title, Description, and Year. Ensure the styling is encapsulated so it

2. Manage State: Create a stateful array called projects. Build a small form that allows a user to input a new project's details and add it to the list.

3.Render Lists using map(): Display the projects array in a grid layout. Each item in the array must be rendered using your ProjectCard component. Requirement: Every list item must have a unique key.

4. Conditional Rendering (Empty State): If the projects array is empty, hide the grid and show a "Getting Started" callout: "You haven't added any projects to your portfolio yet!"

5. Persist Data Locally: Ensure the project list is saved to localStorage. Implement a "Clear Portfolio" button that wipes the state and the local storage simultaneously.

Folder Structure

creator-portfolio/

```
│
├── app/
│   ├── globals.css
│   ├── layout.js
│   └── page.js
│
├── components/
│   └── ProjectCard.jsx
│
├── public/
│
├── package.json
├── next.config.js
└── jsconfig.json
```

1. Reusable Component

components/ProjectCard.jsx

```
"use client";

export default function ProjectCard({ title, description, year }) {
  return (
    <div className="card">
```

```
      <h3>{title}</h3>
      <p>{description}</p>
      <span>{year}</span>
    </div>
  );
}
```

2. Main Page (State, Form, List, LocalStorage)

app/page.js

```
"use client";

import { useEffect, useState } from "react";
import ProjectCard from "../components/ProjectCard";

export default function Home() {
  const [projects, setProjects] = useState([]);
  const [form, setForm] = useState({
    title: "",
    description: "",
    year: ""
  });

  // Load from localStorage
  useEffect(() => {
    const saved = localStorage.getItem("projects");
```

```
  if (saved) {
    setProjects(JSON.parse(saved));
  }
}, []);

// Save to localStorage
useEffect(() => {
  localStorage.setItem("projects", JSON.stringify(projects));
}, [projects]);

const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};

const addProject = (e) => {
  e.preventDefault();

  const newProject = {
    id: Date.now(),
    ...form
  };

  setProjects([...projects, newProject]);

  setForm({ title: "", description: "", year: "" });
```

```
};

const clearPortfolio = () => {
  setProjects([]);
  localStorage.removeItem("projects");
};

return (
  <div className="container">
    <h1>Creator Portfolio</h1>

    {/* FORM */}
    <form onSubmit={addProject} className="form">
      <input
        name="title"
        placeholder="Project Title"
        value={form.title}
        onChange={handleChange}
        required
      />

      <input
        name="description"
        placeholder="Description"
        value={form.description}
```

```
    onChange={handleChange}
    required
   />

   <input
    name="year"
    placeholder="Year"
    value={form.year}
    onChange={handleChange}
    required
   />

   <button type="submit">Add Project</button>
  </form>

  {/* EMPTY STATE */}
  {projects.length === 0 ? (
   <p className="empty">
    You haven't added any projects to your portfolio yet!
   </p>
  ) : (
   <div className="grid">
    {projects.map((project) => (
     <ProjectCard
      key={project.id}
```

```
            title={project.title}
            description={project.description}
            year={project.year}
          />
        ))}
      </div>
    )}

    <button className="clearBtn" onClick={clearPortfolio}>
      Clear Portfolio
    </button>
  </div>
 );
}
```

3. Layout File

app/layout.js

```
import "./globals.css";

export const metadata = {
 title: "Creator Portfolio",
 description: "Portfolio Project"
};

export default function RootLayout({ children }) {
```

```
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

4. Global Styles

app/globals.css

```
/* File: app/globals.css */

*{
  margin:0;
  padding:0;
  box-sizing:border-box;
}

body{
  font-family:Arial, Helvetica, sans-serif;
  background:#f4f7fc;
  padding:40px;
}

.container{
```

```
  max-width:900px;
  margin:auto;
  background:#ffffff;
  padding:30px;
  border-radius:12px;
  box-shadow:0 4px 12px rgba(0,0,0,0.1);
}

h1{
  text-align:center;
  margin-bottom:25px;
}

input{
  width:100%;
  padding:12px;
  margin-bottom:15px;
  border:1px solid #ccc;
  border-radius:8px;
  font-size:16px;
}

.button-group{
  display:flex;
  gap:10px;
```

```
  margin-bottom:20px;
}

button{
  padding:12px 20px;
  border:none;
  border-radius:8px;
  background:#2563eb;
  color:white;
  cursor:pointer;
}

button:hover{
  background:#1d4ed8;
}

.grid{
  display:grid;
  grid-template-columns:repeat(auto-fit,minmax(280px,1fr));
  gap:20px;
}

.card{
  background:white;
  padding:20px;
```

```
  border-radius:10px;
  box-shadow:0 2px 10px rgba(0,0,0,0.08);
}

.card h2{
  color:#2563eb;
  margin-bottom:10px;
}

.card p{
  margin-bottom:10px;
  color:#555;
}

.empty{
  text-align:center;
  margin-top:30px;
  color:gray;
}
```

5. Persist Data Locally:

creator-portfolio/package.json

```
{
  "name": "creator-portfolio",
  "version": "1.0.0",
```

```
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "15.3.3",
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  }
}
```

creator-portfolio/next.config.js

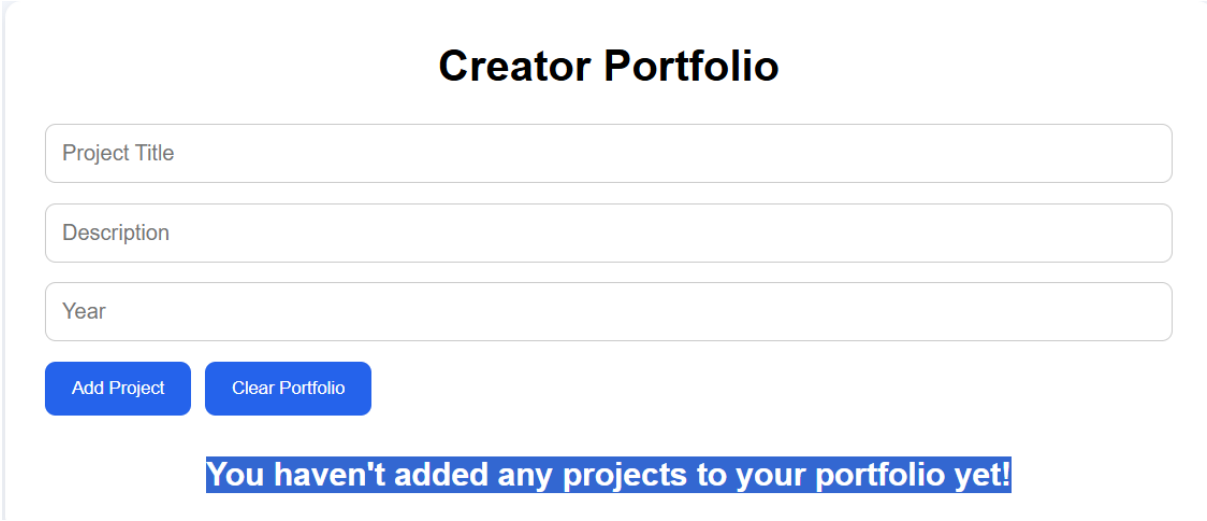
```
const nextConfig = {};

module.exports = nextConfig;
```

creator-portfolio/jsconfig.json

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};
export default config;
```

Output:-


Creator Portfolio
Project Title
Description
Year
Add Project
Clear Portfolio
You haven't added any projects to your portfolio yet!

Section C: Mini Project — Creator "Bio-Stack" Builder

Objective: Build a responsive "Link-in-Bio" app using Next.js and GraphQL to manage a creator's digital identity.

Requirements & Tasks:

1. Add/Edit Profile Details: Create a header section where users can update their Name and Bio. Use a GraphQL Mutation to handle these updates and reflect changes in the UI instantly.

2. Display Projects Dynamically: Implement a GraphQL Query to fetch a list of social links or project cards. Each item must display a Title and a validated URL.

3. Persist Data: Ensure all profile edits and links remain saved after a page refresh. Use localStorage (integrated with your GraphQL cache or state) to maintain data persistence.

4. Clean Component Hierarchy: Organize your code into a clear structure. Separate Layout components (the mobile-first shell) from Data-fetching components and UI Atoms (buttons, inputs).

MINI PROJECT C SECTION

Creator-bio-stack

app/api/graphql/route.ts

```
import { ApolloServer } from "@apollo/server";

import { startServerAndCreateNextHandler } from "@as-integrations/next";

import { makeExecutableSchema } from "@graphql-tools/schema";
```

```
import { typeDefs } from "@/graphql/schema";
import { resolvers } from "@/graphql/resolvers";

const schema = makeExecutableSchema({
  typeDefs,
  resolvers,
});

const server = new ApolloServer({
  schema,
});

const handler = startServerAndCreateNextHandler(server);

export { handler as GET, handler as POST };
```

app/globals.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}

body{
```

```
background:#f5f5f5;

font-family:Arial,Helvetica,sans-serif;

display:flex;

justify-content:center;

padding:30px;

}

.container{

max-width:430px;

width:100%;

background:white;

border-radius:15px;

padding:20px;

box-shadow:0 0 10px rgba(0,0,0,.1);
```

```
}

button{

cursor:pointer;

border:none;

padding:10px;

border-radius:8px;

}

input,textarea{

width:100%;

padding:10px;

margin-top:10px;

margin-bottom:10px;

border:1px solid gray;
```

```
border-radius:8px;

}

.card{

padding:15px;

border:1px solid #ddd;

margin-top:15px;

border-radius:10px;

}

a{

text-decoration:none;

color:blue;

}
```

app/layout.tsx

```
import "./globals.css";
import Providers from "./providers";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

app/page.tsx

```
"use client";

import MobileLayout from "@/components/layout/MobileLayout";
import ProfileHeader from "@/components/profile/ProfileHeader";
import EditProfile from "@/components/profile/EditProfile";
import ProjectList from "@/components/projects/ProjectList";
```

```
export default function Home() {
  return (
    <MobileLayout>
      <ProfileHeader />
      <EditProfile />
      <ProjectList />
    </MobileLayout>
  );
}
```

app/providers.tsx

```
"use client";

export default function Providers({
  children,
}: {
  children: React.ReactNode;
}) {
  return <>{children}</>;
}
```

components/layout/Mobilelayout.tsx

```
import { ReactNode } from "react";

interface Props {
```

```
  children: ReactNode;
}

export default function MobileLayout({ children }: Props) {
  return (
    <div className="container">
      {children}
    </div>
  );
}
```

components/profile/EditProfile.tsx

```
"use client";

import { useState, useEffect } from "react";

import Input from "../ui/Input";
import Button from "../ui/Button";

export default function EditProfile() {
  const [name, setName] = useState("");

  const [bio, setBio] = useState("");

  useEffect(() => {
```

```
  const profile = localStorage.getItem("profile");

  if (profile) {
    const data = JSON.parse(profile);

    setName(data.name);

    setBio(data.bio);
  }
}, []);

const saveProfile = () => {
  const profile = {
    name,
    bio,
  };

  localStorage.setItem(
    "profile",
    JSON.stringify(profile)
  );

  window.location.reload();
};
```

```
  return (
    <div className="card">
      <h2>Edit Profile</h2>

      <Input
        value={name}
        placeholder="Enter Name"
        onChange={(e) => setName(e.target.value)}
      />

      <textarea
        rows={4}
        value={bio}
        placeholder="Enter Bio"
        onChange={(e) => setBio(e.target.value)}
      />

      <Button
        text="Save Profile"
        onClick={saveProfile}
      />
    </div>
  );
}
```

component/profile/ProfileHeader.tsx

```
"use client";

import { useEffect, useState } from "react";

export default function ProfileHeader() {
  const [name, setName] = useState("Creator Name");
  const [bio, setBio] = useState("Welcome to my Bio Stack!");

  useEffect(() => {
    const profile = localStorage.getItem("profile");

    if (profile) {
      const data = JSON.parse(profile);
      setName(data.name);
      setBio(data.bio);
    }
  }, []);

  return (
    <div
      style={{
        textAlign: "center",
        marginBottom: "20px",
      }}
```

```
    >
      <h1>{name}</h1>

      <p
        style={{
          color: "gray",
          marginTop: "10px",
        }}
      >
        {bio}
      </p>
    </div>
  );
}
```

component/projects/ProjectCard.tsx

```
interface Props {
  title: string;
  url: string;
}

export default function ProjectCard({ title, url }: Props) {
  const isValidUrl = (link: string) => {
    try {
```

```
      new URL(link);
      return true;
    } catch {
      return false;
    }
  };

  return (
    <div className="card">
      <h3>{title}</h3>

      {isValidUrl(url) ? (
        <a
          href={url}
          target="_blank"
          rel="noopener noreferrer"
        >
          {url}
        </a>
      ) : (
        <p>Invalid URL</p>
      )}
    </div>
  );
}
```

component/projectsprojectlist.tsx

```
"use client";

import { useEffect, useState } from "react";
import ProjectCard from "./ProjectCard";

interface Project {
  id: number;
  title: string;
  url: string;
}

export default function ProjectList() {
  const defaultProjects: Project[] = [
    {
      id: 1,
      title: "GitHub",
      url: "https://github.com",
    },
    {
      id: 2,
      title: "LinkedIn",
      url: "https://www.linkedin.com",
    },
    {
```

```
    id: 3,
    title: "Portfolio",
    url: "https://example.com",
  },
];

const [projects, setProjects] = useState<Project[]>([]);

useEffect(() => {
  const savedProjects = localStorage.getItem("projects");

  if (savedProjects) {
    setProjects(JSON.parse(savedProjects));
  } else {
    setProjects(defaultProjects);

    localStorage.setItem(
      "projects",
      JSON.stringify(defaultProjects)
    );
  }
}, []);

return (
  <div style={{ marginTop: "20px" }}>
```

```
      <h2>Projects & Social Links</h2>

      {projects.map((project) => (
        <ProjectCard
          key={project.id}
          title={project.title}
          url={project.url}
        />
      ))}
    </div>
  );
}
```

component/ui/Button.tsx

```
interface ButtonProps {
  text: string;
  onClick: () => void;
}

export default function Button({
  text,
  onClick,
}: ButtonProps) {
  return (
```

```
    <button
      onClick={onClick}
      style={{
        width: "100%",
        background: "#2563eb",
        color: "white",
        padding: "12px",
        marginTop: "10px",
        fontSize: "16px",
      }}
    >
      {text}
    </button>
  );
}
```

Components/ui/Input.tsx

```
interface InputProps {
  value: string;
  placeholder: string;
  onChange: (
    e: React.ChangeEvent<HTMLInputElement>
  ) => void;
}
export default function Input({
```

```
  value,
  placeholder,
  onChange,
}: InputProps) {
  return (
    <input
      type="text"
      value={value}
      placeholder={placeholder}
      onChange={onChange}
    />
  );
}
```

creator-bio-stack /graphql/client.ts

```
import {
  ApolloClient,
  InMemoryCache,
  createHttpLink,
} from "@apollo/client";

const link = createHttpLink({
  uri: "/api/graphql",
});
```

```
const client = new ApolloClient({
  link,
  cache: new InMemoryCache(),
});

export default client;
```

creator-bio-stack/graphql/mutations.ts

```
import { gql } from "@apollo/client";

export const UPDATE_PROFILE = gql`
  mutation UpdateProfile(
    $name: String!
    $bio: String!
  ) {
    updateProfile(
      name: $name
      bio: $bio
    ) {
      name
      bio
    }
  }
`;
```

creator-bio-stack/graphql/queries.ts

```
import { gql } from "@apollo/client";

export const GET_PROJECTS = gql`
  query GetProjects {
    projects {
      id
      title
      url
    }
  }
`;

export const GET_PROFILE = gql`
  query GetProfile {
    profile {
      name
      bio
    }
  }
`;
```

creator-bio-stack/graphql/resolvers.ts

```
const profile = {
  name: "Creator Name",
  bio: "Welcome to my Bio Stack",
};

const projects = [
  {
    id: "1",
    title: "GitHub",
    url: "https://github.com",
  },
  {
    id: "2",
    title: "LinkedIn",
    url: "https://linkedin.com",
  },
  {
    id: "3",
    title: "Portfolio",
    url: "https://example.com",
  },
];

export const resolvers = {
```

```
  Query: {
    profile: () => profile,
    projects: () => projects,
  },

  Mutation: {
    updateProfile: (
      _: unknown,
      {
        name,
        bio,
      }: {
        name: string;
        bio: string;
      }
    ) => {
      profile.name = name;
      profile.bio = bio;

      return profile;
    },
  },
};
```

creator-bio-stack/graphql/schemas.ts

```
export const typeDefs = `#graphql

type Profile {
  name: String!
  bio: String!
}

type Project {
  id: ID!
  title: String!
  url: String!
}

type Query {
  profile: Profile!
  projects: [Project!]!
}

type Mutation {
  updateProfile(name: String!, bio: String!): Profile!
}
`;
```

creator-bio-stack/lib/storage.ts

```
export const getProfile = () => {
  if (typeof window === "undefined") return null;

  const data = localStorage.getItem("profile");

  return data ? JSON.parse(data) : null;
};

export const saveProfile = (
  name: string,
  bio: string
) => {
  localStorage.setItem(
    "profile",
    JSON.stringify({
      name,
      bio,
    })
  );
};

export const getProjects = () => {
  if (typeof window === "undefined") return [];
```

```
const data = localStorage.getItem("projects");

if (data) return JSON.parse(data);

const projects = [
  {
    id: 1,
    title: "GitHub",
    url: "https://github.com",
  },
  {
    id: 2,
    title: "LinkedIn",
    url: "https://linkedin.com",
  },
  {
    id: 3,
    title: "Portfolio",
    url: "https://example.com",
  },
];

localStorage.setItem(
  "projects",
  JSON.stringify(projects)
```

```
  );

  return projects;
};
```

creator-bio-stack/type/index.ts

```
export interface Profile {
  name: string;
  bio: string;
}

export interface Project {
  id: number;
  title: string;
  url: string;
}
```

creator-bio-stack/next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  reactStrictMode: true,
};

export default nextConfig;
```

creator-bio-stack/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
```

```
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

creator-bio-stack/packages.json

```
{
  "name": "creator-bio-stack",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@apollo/client": "^4.2.3",
```

```
    "@apollo/client-integration-nextjs": "^0.14.5",
    "@apollo/server": "^5.5.1",
    "@as-integrations/next": "^4.1.0",
    "@graphql-tools/schema": "^10.0.33",
    "graphql": "^16.14.2",
    "next": "16.2.9",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.2.9",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

Final Output:-